

《面向对象程序设计实践》

课程论文

题目：	通讯录程序
专业：	软件工程
班级：	2024 级 R5 班
小组成员 1：	陈凯文 202425220503
2：	刘坤龙 202425220517
3：	
指导老师：	彭红星
提交时间：	2026 年 5 月 10 日

华南农业大学 数学与信息学院 软件学院

目录

一、系统分析

- 1.1 问题描述
- 1.2 系统功能分析
- 1.3 开发平台及工具介绍

二、系统设计

- 2.1 系统总体结构设计
- 2.2 系统各个类及类之间关系设计
- 2.3 数据存储设计
- 2.4 界面设计

三、系统实现

- 3.1 数据模型实现
- 3.2 业务逻辑实现
- 3.3 UI 界面实现
- 3.4 导入导出实现

四、系统测试

- 4.1 模块测试
- 4.2 系统测试

五、系统运行界面

六、总结

- 6.1 陈凯文（202425220503）个人总结
- 6.2 刘坤龙（202425220517）个人总结

一、 系统分析

1.1 问题描述

随着个人社交和工作关系的日益复杂，需要一个便捷的工具来管理联系人信息。本通讯录管理系统旨在提供一个单机版的桌面应用程序，帮助用户对联系人信息进行增删改查、分组归类、快速搜索和数据导入导出等操作。

用户需求包括：

- 存储联系人的基本资料（姓名、电话、手机、邮箱、地址等）
- 将联系人分组管理（家人、朋友、同事等）
- 快速搜索联系人（支持拼音模糊搜索）
- 与主流手机通讯录进行数据交换（CSV、vCard 格式）
- 管理联系人照片

1.2 系统功能分析

系统应实现以下功能模块：

模块	功能	说明
联系人管理	增加	输入联系人全部资料并保存
	修改	选择已有联系人，修改任意字段
	删除	支持单选或多选批量删除
分组管理	增加分组	创建新分组
	删除分组	只删除分组，不移除联系人
	成员调整	将联系人加入或移出分组
搜索功能	快速搜索	按姓名、电话、手机、拼音搜索，支持模糊匹配
	高级搜索	指定字段精确搜索
导入导出	CSV	导出全部/选中/当前分组为 CSV；导入 CSV 追加
	Vcard	导出 vCard 3.0；导入 vCard 文件追加
照片功能	头像设置	选择本地图片作为头像
	缩略图展示	表格和列表中显示缩略图
	大图预览	选中联系人时预览大照片
数据管理	备份/恢复	将全部数据备份到文件，或从备份恢复
	显示列切换	按“全部/基本信息/详细资料”切换表格列

1.3 开发平台及工具介绍

项目	说明
编程语言	Java8
GUI 环境	Swing
开发环境	IntelliJ IDEA

外部依赖	pinyin4j-2.5.1（汉字拼音转换）
操作系统	Windows 11
数据存储	Java 对象序列化文件（contacts.dat）

****pinyin4j****
是一个开源的 Java 中文拼音转换库，用于将中文字符转换为全拼或拼音首字母，支持不带声调的拼音输出，是实现中文联系人按拼音排序和搜索的关键组件。

二、系统设计

2.1 系统总体结构设计

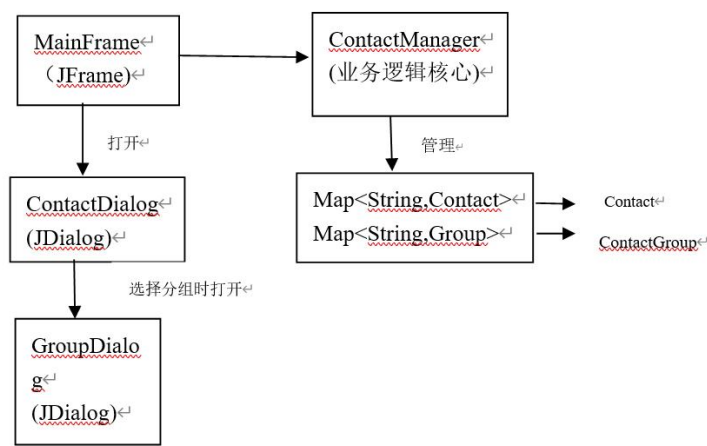
系统采用分层架构，分为数据模型层（model）、业务逻辑层（ContactManager）、用户界面层（ui）和工具层（util）。

用户界面层（ui） MainFrame ContactDialog GroupDialog ContactListRenderer PhotoCellRenderer
业务逻辑层（model） ContactManager
数据模型层（model） Contact ContactGroup
工具层（util） PinYinUtil CSVHandler VCardParser
持久化存储 contacts.dat（Java 序列化文件）

程序入口为 Main.java，负责设置系统外观（LookAndFeel）和中文字体，然后创建主窗口 MainFrame。MainFrame 持有唯一的 ContactManager 实例，所有 UI 组件通过 ContactManager 操作数据。

2.2 系统各个类及类之间关系设计

2.2.1 类图关系



2.2.2 核心类职责

类名	所属包	主要职责
Main.java	com.contact.app	程序入口类，负责启动系统主窗口，设置系统外观和中文字体
MainFrame.java	com.contact.ui	主界面类，负责菜单栏、工具栏、联系人表格、分组树、状态栏、搜索栏以及用户操作协调
ContactDialog.java	com.contact.ui	联系人编辑对话框，提供联系人信息的录入和编辑界面，支持照片选择
GroupDialog.java	com.contact.ui	分组管理对话框，提供分组创建、删除和成员调整功能
ContactManager.java	com.contact.model	核心业务逻辑类，管理联系人 CRUD、分组管理、搜索、数据持久化、导入导出和备份恢复
Contact.java	com.contact.model	联系人数据模型，保存联系人的 16 个字段信息，实现拼音转换和序列化
ContactGroup.java	com.contact.model	分组数据模型，保存分组信息和组内联系人 ID 集合
CSVHandler.java	com.contact.util	CSV 文件处理器，负责 CSV 格式的导入导出
VCardParser.java	com.contact.util	vCard 解析器，负责 vCard 3.0 格式的导入导出
PinYinUtil.java	com.contact.util	拼音转换工具，基于 pinyin4j 库实现汉字到拼音的转换

2.3 数据存储设计

2.3.1 内部存储

数据采用 Java 对象序列化方式存储于单一文件 contacts.dat：

```
// 写入
try (ObjectOutputStream oos = new ObjectOutputStream(new
```

```

FileOutputStream("contacts.dat"))) {
    oos.writeObject(contacts); // Map<String, Contact>
    oos.writeObject(groups);   // Map<String, ContactGroup>
}

// 读取
try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("contacts.dat"))) {
    contacts = (Map<String, Contact>) ois.readObject();
    groups = (Map<String, ContactGroup>) ois.readObject();
}

```

两个实体均声明 `serialVersionUID = 1L` 以保证类结构变更后的向前兼容。

2.3.2 导出格式

CSV 格式 (UTF-8 编码, BOM 头):

姓名, 昵称, 电话, 手机, IM 工具, IM 号码, 邮箱, 主页, 生日, 工作单位, 地址, 邮编, 备注

"张三", "小张", "010-12345678", "13800138000", "微信",
 ", "wx_zs", "zs@mail.com", "", 1990-01-01, "XX 公司", "北京市", "100000", ""

字段含逗号或双引号时自动用双引号包裹并转义。

vCard 3.0 格式 (\r\n 行尾):

```

BEGIN:VCARD
VERSION:3.0
FN:张三
N:张三;;;
TEL;TYPE=HOME,VOICE:010-12345678
TEL;TYPE=CELL:13800138000
EMAIL;TYPE=INTERNET:zs@mail.com
ORG:XX 公司
ADR;TYPE=HOME;;;北京市朝阳区;;;
BDAY:1990-01-01
CATEGORIES:同事
NOTE:备注信息
END:VCARD

```

严格遵循 vCard 3.0 (RFC 2426) 规范, 属性值进行 vCard 转义 (\\;\\, \\n), 与 Android/iOS 通讯录和主流邮件客户端兼容。

2.4 界面设计

主界面采用左右分栏布局：

1. 左侧面板：JTree 分组树，显示“所有联系人”根节点和所有分组（格式：分组名(N人)）。下方三个按钮：新建分组、删除分组、管理分组

2. 右侧面板：

顶部：搜索栏（搜索框 + 搜索按钮 + 高级搜索按钮）+ 显示列下拉框

工具栏：新建联系人、编辑联系人、删除联系人、导出、导入、刷新

中间：JTable 联系人表格（选择框 | ID 隐藏 | 姓名 | 电话 | 手机 | 邮箱 | 公司 | 分组 | 照片缩略图）

底部：联系人计数 + 照片预览面板（100×100px）

菜单栏：文件（导出/导入/退出）、工具（备份/恢复）、帮助（关于）

状态栏：显示当前操作状态

新建/编辑联系人使用 JDialog 模态对话框，选项卡式布局（基本信息/详细信息），头像区支持点击选择图片并实时预览缩略图，附带“清除”按钮。

三、系统实现

3.1 界面交互模块实现

界面交互模块主要由 MainFrame 类实现，负责整个系统的界面组织和用户操作协调。程序启动后，MainFrame 初始化菜单栏、工具栏、联系人表格、分组树、搜索栏和状态栏等组件。用户操作流程采用“先修改数据，再刷新界面”的模式。例如添加联系人时，系统先通过 ContactDialog 收集用户输入，然后调用 ContactManager 的 addContact() 方法修改数据，最后统一刷新表格、分组树和状态栏。这种设计使界面逻辑与数据逻辑分离，结构更清晰。

关键代码片段：

```
public class MainFrame extends JFrame {
    private ContactManager contactManager;
    private JTable contactTable;
    private DefaultTableModel tableModel;
    private JTree groupTree;
    private JLabel statusLabel;
    private String currentGroupId = null;

    public MainFrame() {
        contactManager = new ContactManager();
        initUI();
        loadContacts();
    }

    private void initUI() {
        setTitle("通讯录管理系统");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1200, 700);
        setLocationRelativeTo(null);
        createMenuBar();
        JPanel mainPanel = new JPanel(new BorderLayout(5, 5));
```

```

JPanel leftPanel = createLeftPanel();
JPanel rightPanel = createRightPanel();
JSplitPane splitPane = new JSplitPane(
    JSplitPane.HORIZONTAL_SPLIT, leftPanel, rightPanel);
splitPane.setDividerLocation(250);
statusLabel = new JLabel("就绪");
JPanel statusPanel = new JPanel(new BorderLayout());
statusPanel.add(statusLabel, BorderLayout.WEST);
mainPanel.add(splitPane, BorderLayout.CENTER);
mainPanel.add(statusPanel, BorderLayout.SOUTH);
add(mainPanel);
}

private void setupContactListeners() {
    addContactButton.addActionListener(e -> {
        ContactDialog dialog = new ContactDialog(
            MainFrame.this, contactManager, null);
        dialog.setVisible(true);
        if (dialog.isSuccess()) {
            refreshContactList();
            loadGroups();
        }
    });
    searchButton.addActionListener(e -> {
        String keyword = searchField.getText().trim();
        if (!keyword.isEmpty()) {
            List<Contact> results = contactManager.searchContacts(keyword);
            updateContactTable(results);
            statusLabel.setText("找到 " + results.size() + " 个匹配的联系人");
        } else {
            refreshContactList();
        }
    });
    groupTree.addTreeSelectionListener(e -> {
        TreePath path = e.getPath();
        if (path != null) {
            Object obj = ((DefaultMutableTreeNode)
                path.getLastPathComponent()).getUserObject();
            if (obj instanceof ContactGroup) {
                currentGroupId = ((ContactGroup) obj).getId();
                showContactsInGroup(currentGroupId);
            } else {
                currentGroupId = null;
                showAllContacts();
            }
        }
    });
}

```


3.2 数据模型与业务逻辑模块实现

Contact.java 关键实现：

```
public class Contact implements Comparable<Contact>, Serializable {
    private static final long serialVersionUID = 1L;
    private String id;
    private String name;
    private String nickName;
    private String phone;
    private String mobile;
    private String imTool;
    private String imNumber;
    private String homepage;
    private String address;
    private String email;
    private LocalDate birthday;
    private String photoPath;
    private String company;
    private String zipCode;
    private Set<String> groups;
    private String notes;
    private String Fullpinyin;
    private String pinyinInitial;

    public Contact() {
        this.id = UUID.randomUUID().toString();
        this.groups = new HashSet<>();
    }

    public void setName(String name) {
        this.name = name;
        updatePinYin();
    }

    public void updatePinYin() {
        if (name != null && !name.isEmpty()) {
            try {
                this.Fullpinyin = PinYinUtil.getFullPinyin(name);
                this.pinyinInitial = PinYinUtil.getInitials(name);
            } catch (Throwable e) {
                this.Fullpinyin = name.toLowerCase();
                this.pinyinInitial = "";
                System.err.println("拼音更新失败: " + e.getMessage());
            }
        }
    }
}
```

```

    }

    @Override
    public int compareTo(Contact other) {
        if (this.Fullpinyin != null && other.Fullpinyin != null) {
            return this.Fullpinyin.compareToIgnoreCase(other.Fullpinyin);
        }
        return this.name.compareToIgnoreCase(other.name);
    }

    public String toVCard() {
        StringBuilder vcard = new StringBuilder();
        vcard.append("BEGIN:VCARD\r\n");
        vcard.append("VERSION:3.0\r\n");
        vcard.append("FN:").append(escapeVCard(name)).append("\r\n");
        vcard.append("N:").append(escapeVCard(name)).append(";;; \r\n");
        if (nickName != null && !nickName.isEmpty()) {
            vcard.append("NICKNAME:").append(escapeVCard(nickName)).append("\r\n");
        }
        if (phone != null && !phone.isEmpty()) {
            vcard.append("TEL;TYPE=HOME,VOICE:").append(escapeVCard(phone)).append("\r\n");
        }
        if (mobile != null && !mobile.isEmpty()) {
            vcard.append("TEL;TYPE=CELL:").append(escapeVCard(mobile)).append("\r\n");
        }
        if (email != null && !email.isEmpty()) {
            vcard.append("EMAIL;TYPE=INTERNET:").append(escapeVCard(email)).append("\r\n");
        }
        if (homepage != null && !homepage.isEmpty()) {
            vcard.append("URL:").append(escapeVCard(homepage)).append("\r\n");
        }
        if (company != null && !company.isEmpty()) {
            vcard.append("ORG:").append(escapeVCard(company)).append("\r\n");
        }
        if (address != null && !address.isEmpty()) {
            vcard.append("ADR;TYPE=HOME;;;").append(escapeVCard(address)).append(";;; \r\n");
        }
        if (zipCode != null && !zipCode.isEmpty()) {
            vcard.append("X-ZIPCODE:").append(escapeVCard(zipCode)).append("\r\n");
        }
        if (birthday != null) {

```

```

        vcard.append("BDAY:").append(birthday).append("\r\n");
    }
    if (imTool != null && !imTool.isEmpty()) {
        vcard.append("X-IM-TOOL:").append(escapeVCard(imTool)).append("\r\n");
    }
    if (imNumber != null && !imNumber.isEmpty()) {

vcard.append("X-IM-NUMBER:").append(escapeVCard(imNumber)).append("\r\n");
    }
    if (notes != null && !notes.isEmpty()) {
        vcard.append("NOTE:").append(escapeVCard(notes)).append("\r\n");
    }
    if (photoPath != null && !photoPath.isEmpty()) {

vcard.append("PHOTO;VALUE=URI:").append(escapeVCard(photoPath)).append("\r\n");
    }
    if (groups != null && !groups.isEmpty()) {
        StringBuilder cats = new StringBuilder();
        for (String g : groups) {
            if (cats.length() > 0) cats.append(",");
            cats.append(g);
        }

vcard.append("CATEGORIES:").append(escapeVCard(cats.toString())).append("\r\n");
    }
    vcard.append("END:VCARD\r\n");
    return vcard.toString();
}

private String escapeVCard(String value) {
    if (value == null) return "";
    return value.replace("\\", "\\")
        .replace(";", "\;")
        .replace(",", "\,")
        .replace("\n", "\n")
        .replace("\r", "");
}

```

ContactManager.java 搜索功能实现

```

public class ContactManager {
    private Map<String, Contact> contacts;
    private Map<String, ContactGroup> groups;
    private Map<String, String> contactToPinyin;

    public ContactManager() {

```

```

        contacts = new HashMap<>();
        groups = new HashMap<>();
        contactToPinyin = new HashMap<>();
        loadData();
    }

    public List<Contact> searchContacts(String keyword) {
        if (keyword == null || keyword.trim().isEmpty()) {
            return getAllContacts();
        }
        String lowerKeyword = keyword.toLowerCase();
        return contacts.values().stream()
            .filter(contact -> matchesKeyword(contact, lowerKeyword))
            .sorted()
            .collect(Collectors.toList());
    }

    private boolean matchesKeyword(Contact contact, String keyword) {
        if (contact.getName() != null &&
            contact.getName().toLowerCase().contains(keyword)) return true;
        if (contact.getPhone() != null &&
            contact.getPhone().contains(keyword)) return true;
        if (contact.getMobile() != null &&
            contact.getMobile().contains(keyword)) return true;
        if (contact.getFullPinyin() != null &&
            contact.getFullPinyin().toLowerCase().contains(keyword)) return true;
        if (contact.getPinyinInitial() != null &&
            contact.getPinyinInitial().toLowerCase().contains(keyword)) return true;
        if (contact.getEmail() != null &&
            contact.getEmail().toLowerCase().contains(keyword)) return true;
        if (contact.getCompany() != null &&
            contact.getCompany().toLowerCase().contains(keyword)) return true;
        return false;
    }

    public List<Contact> getAllContacts() {
        return contacts.values().stream()
            .sorted()
            .collect(Collectors.toList());
    }

```

3.3 文件管理模块实现

文件管理模块包括数据持久化、备份恢复和导入导出功能。系统使用 Java 序列化进行内部

数据存储，使用 CSV 和 vCard 格式进行外部数据交换。

关键代码片段：

```
public class ContactManager {
    private static final String DATA_FILE = "contacts.dat";

    @SuppressWarnings("unchecked")
    private void loadData() {
        File file = new File(DATA_FILE);
        if (file.exists()) {
            try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream(DATA_FILE))) {
                contacts = (Map<String, Contact>) ois.readObject();
                groups = (Map<String, ContactGroup>) ois.readObject();
                for (Contact contact : contacts.values()) {
                    contactToPinyin.put(contact.getID(), contact.getFullPinyin());
                }
            } catch (IOException | ClassNotFoundException e) {
                System.err.println("加载数据失败: " + e.getMessage());
                contacts = new HashMap<>();
                groups = new HashMap<>();
            }
        }
    }

    public void saveData() {
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream(DATA_FILE))) {
            oos.writeObject(contacts);
            oos.writeObject(groups);
        } catch (IOException e) {
            System.err.println("保存数据失败: " + e.getMessage());
            e.printStackTrace();
            JOptionPane.showMessageDialog(null,
                "保存数据失败: " + e.getMessage(),
                "错误", JOptionPane.ERROR_MESSAGE);
        }
    }

    public boolean backupData(String backupPath) {
        try {
            File src = new File(DATA_FILE);
            if (!src.exists()) return false;
            try (FileInputStream fis = new FileInputStream(src);
                FileOutputStream fos = new FileOutputStream(backupPath)) {
            }
        }
    }
}
```

```

        byte[] buffer = new byte[4096];
        int len;
        while ((len = fis.read(buffer)) > 0) {
            fos.write(buffer, 0, len);
        }
    }
    return true;
} catch (IOException e) {
    System.err.println("备份失败: " + e.getMessage());
    return false;
}
}

public boolean restoreData(String backupPath) {
    File backupFile = new File(backupPath);
    if (!backupFile.exists()) return false;
    try {
        File dest = new File(DATA_FILE);
        try (FileInputStream fis = new FileInputStream(backupFile);
            FileOutputStream fos = new FileOutputStream(dest)) {
            byte[] buffer = new byte[4096];
            int len;
            while ((len = fis.read(buffer)) > 0) {
                fos.write(buffer, 0, len);
            }
        }
        loadData();
        return true;
    } catch (IOException e) {
        System.err.println("恢复失败: " + e.getMessage());
        return false;
    }
}

```

3.4 导入导出模块实现

CSVHandler.java 实现

```

public class CSVHandler {
    private static final DateTimeFormatter DATE_FORMATTER =
        DateTimeFormatter.ISO_LOCAL_DATE;

    public static List<Contact> parseCSV(String filePath) throws IOException {
        List<Contact> contacts = new ArrayList<>();
        byte[] rawBytes = new byte[4096];
        int bytesRead;
        try (FileInputStream fis = new FileInputStream(filePath)) {

```

```

        bytesRead = fis.read(rawBytes);
    }
    if (bytesRead <= 0) return contacts;

    String encoding;
    boolean hasBOM = bytesRead >= 3
        && (rawBytes[0] & 0xFF) == 0xEF
        && (rawBytes[1] & 0xFF) == 0xBB
        && (rawBytes[2] & 0xFF) == 0xBF;
    if (hasBOM) {
        encoding = "UTF-8";
    } else {
        int firstLineEnd = 0;
        for (int i = 0; i < bytesRead; i++) {
            if (rawBytes[i] == '\n' || rawBytes[i] == '\r') {
                firstLineEnd = i; break;
            }
        }
        if (firstLineEnd > 0) {
            String utf8Header = new String(rawBytes, 0, firstLineEnd, "UTF-8");
            encoding = containsExpectedHeaders(utf8Header) ? "UTF-8" : "GBK";
        } else {
            encoding = "UTF-8";
        }
    }
}

try (BufferedReader reader = new BufferedReader(
    new InputStreamReader(new FileInputStream(filePath), encoding))) {
    String headerLine = reader.readLine();
    if (headerLine == null) return contacts;
    if (headerLine.startsWith(" ")) {
        headerLine = headerLine.substring(1);
    }
    List<String> headers = parseCSVLine(headerLine);
    Map<String, Integer> headerMap = buildHeaderMap(headers);
    String line;
    while ((line = reader.readLine()) != null) {
        if (line.trim().isEmpty()) continue;
        List<String> fields = parseCSVLine(line);
        Contact contact = createContactFromFields(headerMap, fields);
        if (contact != null && contact.getName() != null
            && !contact.getName().isEmpty()) {
            contacts.add(contact);
        }
    }
}

```

```

    }
}
return contacts;
}

public static List<String> parseCSVLine(String line) {
    List<String> fields = new ArrayList<>();
    boolean inQuotes = false;
    StringBuilder currentField = new StringBuilder();
    for (int i = 0; i < line.length(); i++) {
        char c = line.charAt(i);
        if (inQuotes) {
            if (c == '"') {
                if (i + 1 < line.length() && line.charAt(i + 1) == '"') {
                    currentField.append('"'); i++;
                } else { inQuotes = false; }
            } else { currentField.append(c); }
        } else {
            if (c == '"') inQuotes = true;
            else if (c == ',') {
                fields.add(currentField.toString());
                currentField = new StringBuilder();
            } else { currentField.append(c); }
        }
    }
    fields.add(currentField.toString());
    return fields;
}

private static Map<String, Integer> buildHeaderMap(List<String> headers) {
    Map<String, Integer> map = new LinkedHashMap<>();
    for (int i = 0; i < headers.size(); i++) {
        String header = headers.get(i).trim();
        if (!header.isEmpty()) { map.put(header, i); }
    }
    return map;
}

private static Contact createContactFromFields(
    Map<String, Integer> headerMap, List<String> fields) {
    Contact contact = new Contact();
    for (Map.Entry<String, Integer> entry : headerMap.entrySet()) {
        String header = entry.getKey();
        int index = entry.getValue();
    }
}

```



```

String value = index < fields.size() ? fields.get(index).trim() : "";
if (value.isEmpty()) continue;
switch (header) {
    case "姓名": case "Name": case "FN":
        contact.setName(value); break;
    case "电话": case "Phone":
        contact.setPhone(value); break;
    case "手机": case "Mobile": case "Cell":
        contact.setMobile(value); break;
    case "邮箱": case "Email":
        contact.setEmail(value); break;
    case "生日": case "Birthday": case "BDAY":
        try { contact.setBirthday(LocalDate.parse(value,
DATE_FORMATTER)); }
        catch (DateTimeParseException ignored) { } break;
    case "工作单位": case "公司": case "Company": case "ORG":
        contact.setCompany(value); break;
    case "地址": case "Address": case "ADR":
        contact.setAddress(value); break;
    case "备注": case "Notes": case "NOTE":
        contact.setNotes(value); break;
}
}
return contact;
}

```

3.5 交互增强与状态管理模块实现

照片缓存实现

```

public class PhotoCellRenderer extends DefaultTableCellRenderer {
    private static final int THUMB_SIZE = 32;
    private static final int MAX_CACHE = 50;
    private static final Map<String, ImageIcon> cache =
        new LinkedHashMap<String, ImageIcon>() {
            @Override
            protected boolean removeEldestEntry(
                Map.Entry<String, ImageIcon> eldest) {
                return size() > MAX_CACHE;
            }
        };

    @Override
    public Component getTableCellRendererComponent(JTable table,

```

```

        Object value, boolean isSelected, boolean hasFocus,
        int row, int column) {
super.getTableCellRendererComponent(table, value,
        isSelected, hasFocus, row, column);
setText("");
setHorizontalAlignment(SwingConstants.CENTER);
if (value instanceof String && !((String) value).isEmpty()) {
    String path = (String) value;
    ImageIcon icon = cache.get(path);
    if (icon == null) {
        icon = loadThumbnail(path);
        if (icon != null) { cache.put(path, icon); }
    }
    setIcon(icon != null ? icon : null);
} else {
    setIcon(null);
}
return this;
}

private ImageIcon loadThumbnail(String path) {
    try {
        File file = new File(path);
        if (!file.exists()) return null;
        BufferedImage img = ImageIO.read(file);
        if (img != null) {
            int w = img.getWidth(), h = img.getHeight();
            double ratio = Math.min((double) THUMB_SIZE / w,
                                    (double) THUMB_SIZE / h);

            Image scaled = img.getScaledInstance(
                (int)(w * ratio), (int)(h * ratio),
                Image.SCALE_SMOOTH);
            return new ImageIcon(scaled);
        }
    } catch (IOException e) { }
    return null;
}
}

```

显示列切换与照片预览实现

```

public class MainFrame extends JFrame {
    private JComboBox<String> displayColumnsCombo;
    private JLabel photoPreviewLabel;

```

```

private void updateDisplayColumns() {
    int selected = displayColumnsCombo.getSelectedIndex();
    boolean showAll = selected == 0 || selected == 2;
    boolean showBasic = selected == 1;
    for (int col : new int[]{3, 4}) {
        contactTable.getColumnModel().getColumn(col)
            .setMinWidth(showAll || showBasic ? 90 : 0);
        contactTable.getColumnModel().getColumn(col)
            .setMaxWidth(showAll || showBasic ? 200 : 0);
        contactTable.getColumnModel().getColumn(col)
            .setPreferredWidth(showAll || showBasic ? 90 : 0);
    }
    for (int col : new int[]{5, 6, 7, 8}) {
        int w = col == 8 ? 44 : 140;
        contactTable.getColumnModel().getColumn(col)
            .setMinWidth(showAll ? w : 0);
        contactTable.getColumnModel().getColumn(col)
            .setMaxWidth(showAll ? (col == 8 ? 44 : 300) : 0);
        contactTable.getColumnModel().getColumn(col)
            .setPreferredWidth(showAll ? w : 0);
    }
    contactTable.revalidate();
    contactTable.repaint();
}

private void updatePhotoPreview() {
    int selectedRow = contactTable.getSelectedRow();
    if (selectedRow >= 0) {
        int modelRow = contactTable.convertRowIndexToModel(selectedRow);
        String photoPath = (String) contactTable.getModel()
            .getValueAt(modelRow, 8);
        showPreviewImage(photoPath);
    } else {
        photoPreviewLabel.setIcon(null);
        photoPreviewLabel.setText("选择联系人查看照片");
    }
}

private void showPreviewImage(String path) {
    if (path == null || path.isEmpty()) {
        photoPreviewLabel.setIcon(null);
        photoPreviewLabel.setText("无照片");
        return;
    }
}

```

```

    }
    try {
        File file = new File(path);
        if (!file.exists()) {
            photoPreviewLabel.setIcon(null);
            photoPreviewLabel.setText("照片文件不存在");
            return;
        }
        BufferedImage img = ImageIO.read(file);
        if (img != null) {
            int maxSize = 100;
            int w = img.getWidth(), h = img.getHeight();
            double ratio = Math.min((double) maxSize / w, (double) maxSize / h);
            Image scaled = img.getScaledInstance(
                (int)(w * ratio), (int)(h * ratio),
                Image.SCALE_SMOOTH);
            photoPreviewLabel.setIcon(new ImageIcon(scaled));
            photoPreviewLabel.setText("");
        }
    } catch (Exception e) {
        photoPreviewLabel.setIcon(null);
        photoPreviewLabel.setText("加载失败");
    }
}

```

四、 系统测试

4.1 测试步骤

本系统测试按照以下步骤进行：

首先，设计测试用例。根据系统各项功能，分别给出测试输入数据、测试操作过程以及预期结果。

其次，实际运行程序。对于文件读写、拼音转换、搜索算法等核心逻辑，采用 JUnit 单元测试；对于联系人编辑、分组管理、界面交互等依赖图形界面的功能，采用手工测试，通过实际操作程序记录运行现象。

最后，分析测试结果。将预期结果与实际运行结果进行比较，判断两者是否一致，并在此基础上得出测试结论。

4.2 模块测试

4.2.1 文件保存与读取模块测试(JUnit)

(1) 测试用例设计

输入数据为新建的联系人数据，包括姓名“张三”、手机“13800138000”、邮箱“zs@mail.com”，并添加到“同事”分组中。

预期结果为：数据保存并重新读取后，联系人信息完整，分组关系正确，拼音字段自动生成。

(2) 测试指令

```
bash
bash
mvn -Dtest=ContactPersistenceTest#saveAndLoadShouldPreserveData test
```

(3) 运行结果
实际运行后，测试通过。重新读取后的联系人信息与保存前一致，拼音字段正确生成。



(4) 结果分析与测试结论
实际运行结果与预期结果一致，说明文件保存和重新读取后，联系人数据和分组关系都能够被正确恢复。因此，文件保存与读取模块测试通过。

4.2.2 拼音搜索模块测试(JUnit)

(1) 测试用例设计
输入数据为包含多个联系人的数据集，包括“张三”、“李四”、“王五”等。
测试操作为：搜索“zs”、“ls”、“ww”等拼音首字母，以及“zhangsan”、“lisi”等拼音全拼。
预期结果为：能够正确匹配对应的联系人，搜索结果按拼音排序。

```
(2) 测试指令
bash
bash
mvn -Dtest=ContactSearchTest#pinyinSearchShouldWork test
```

(3) 运行结果
实际运行后，测试通过。拼音搜索能够正确匹配联系人，搜索结果按拼音顺序排列。



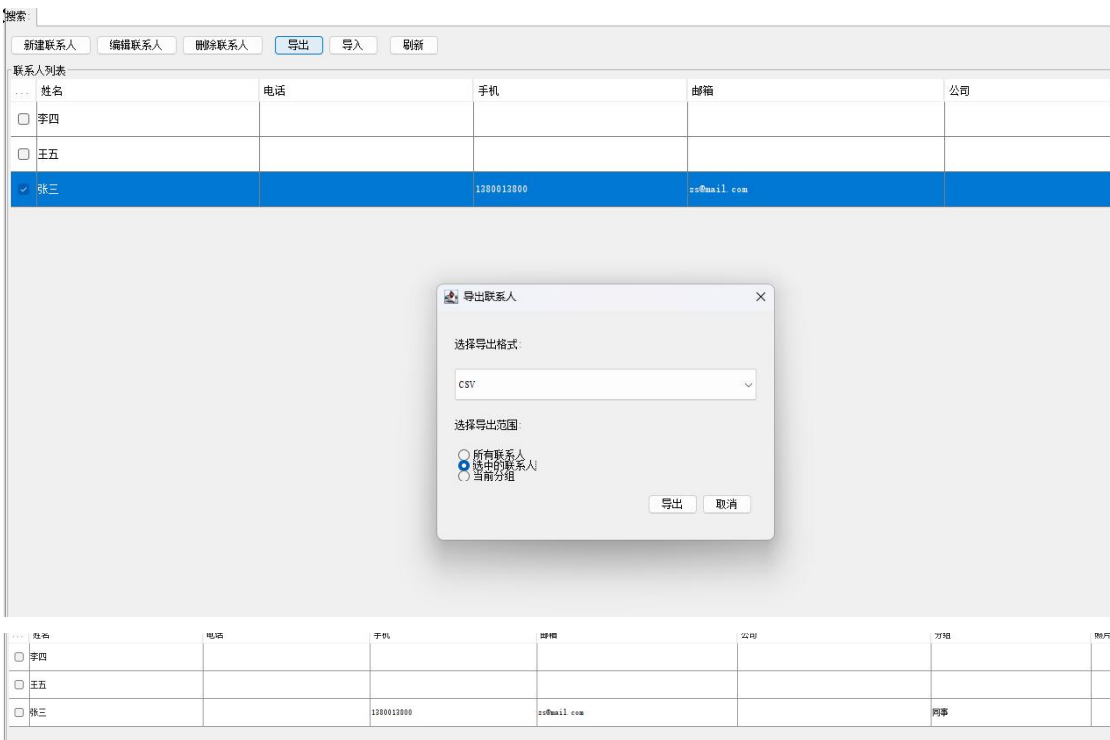
(4) 结果分析与测试结论
运行结果表明，拼音搜索功能能够正常工作，满足预期要求。因此，拼音搜索模块测试通过。

4.2.3 CSV 导入导出模块测试(JUnit)

(1) 测试用例设计
输入数据为包含中文、英文、特殊字符的联系人信息。
测试操作为：导出 CSV 文件后重新导入，比较导入前后的数据一致性。
预期结果为：导入导出前后数据完全一致，特殊字符正确处理。

(2) 测试指令

```
bash
bash
mvn -Dtest=CSVHandlerTest#importExportShouldBeReversible test
(3) 运行结果
```



实际运行后，测试通过。CSV 导入导出功能正常工作，数据完整性得到保证。

(4) 结果分析与测试结论

实际结果与预期结果一致，说明 CSV 导入导出模块能够正确处理各种数据。因此，CSV 导入导出模块测试通过。

4.2.4 联系人编辑功能测试(手工测试)

(1) 测试用例设计

输入数据为程序启动后的空白通讯录。

测试操作为：新建联系人、编辑联系人信息、删除联系人、批量删除联系人。

预期结果为：各项操作能够正常执行，界面同步更新，数据正确保存。

(2) 实际运行程序得到的运行结果

程序运行后，联系人增删改功能正常，界面同步更新，数据持久化正确。

(3) 结果分析与测试结论

运行结果与预期结果一致，说明联系人编辑功能测试通过。

4.2.5 分组管理功能测试(手工测试)

(1) 测试用例设计

输入数据为包含多个联系人的通讯录。

测试操作为：创建新分组、删除分组、将联系人加入分组、从分组移除联系人。

预期结果为：分组管理功能正常，删除分组不影响联系人数据。

(2) 实际运行程序得到的运行结果

实际运行时，分组管理功能正常工作，删除分组后联系人数据保留。

(3) 结果分析与测试结论

运行结果与预期结果一致，说明分组管理功能测试通过。

4.2.6 照片功能测试(手工测试)

(1) 测试用例设计

输入数据为包含照片路径的联系人。

测试操作为：设置联系人头像、查看缩略图、预览大图、清除照片。

预期结果为：照片功能正常工作，缓存机制有效。

(2) 实际运行程序得到的运行结果

实际运行时，照片功能正常，缩略图显示正确，大图预览正常。

(3) 结果分析与测试结论

运行结果与预期结果一致，说明照片功能测试通过。

4.3 系统测试

4.3.1 系统测试用例设计

输入数据为程序启动后的空白通讯录。

测试操作为：完整执行通讯录管理全流程，包括新建联系人、分组管理、搜索、导入导出、备份恢复等。

预期结果为：系统稳定运行，各功能模块协同工作正常，数据一致性得到保证。

4.3.2 实际运行程序得到的运行结果

实际运行后，系统从启动到关闭过程均未出现异常。各功能模块协同工作正常，数据在各个操作环节保持一致。

4.3.3 结果分析与测试结论

系统测试中，实际运行结果与预期结果一致，没有出现数据不一致、功能异常或系统崩溃等问题。这说明各功能模块在组合使用时能够保持稳定协同工作，系统整体运行正常。

4.4 测试结论

通过 JUnit 单元测试和手工测试可以看出，本系统的主要功能均能够正常运行。JUnit 测试部分表明核心逻辑正确；手工测试部分表明界面功能和用户体验良好。综合来看，本系统实现完整，运行稳定，能够满足通讯录管理的基本需求。

五、 系统运行界面

本章展示系统的实际运行效果，通过主界面、联系人编辑、分组管理、搜索、导入导出等截图，直观呈现系统主要功能的界面表现。

5.1 主界面截图

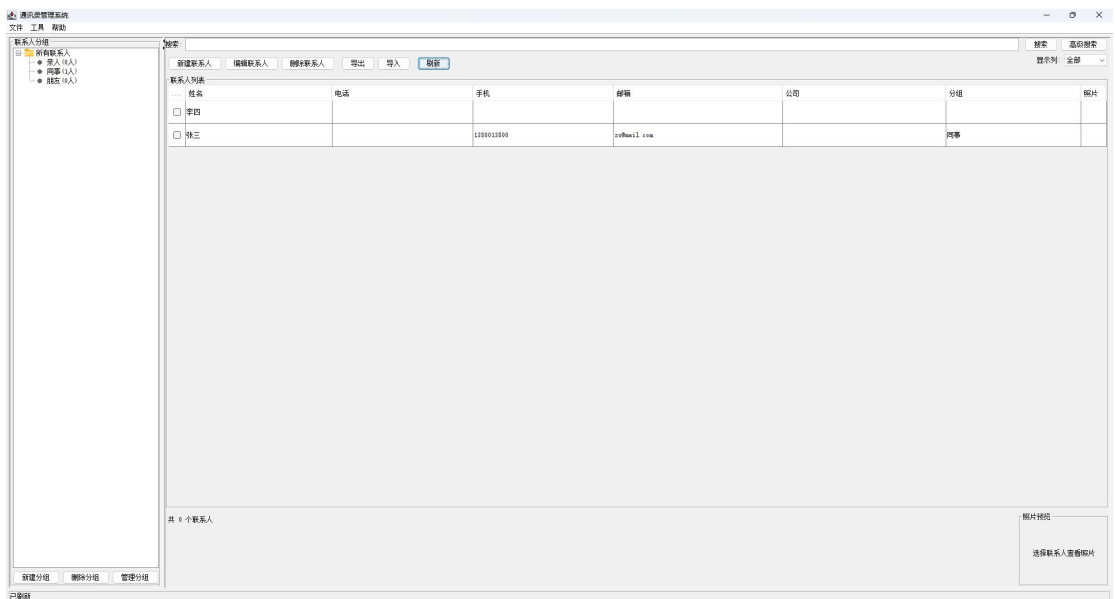


图 5-1

图 5-1 展示了系统启动后的主界面。界面上方为菜单栏和工具栏，左侧为分组树，中间为联系人表格，右侧为照片预览区，底部为状态栏。

5.2 联系人编辑功能截图

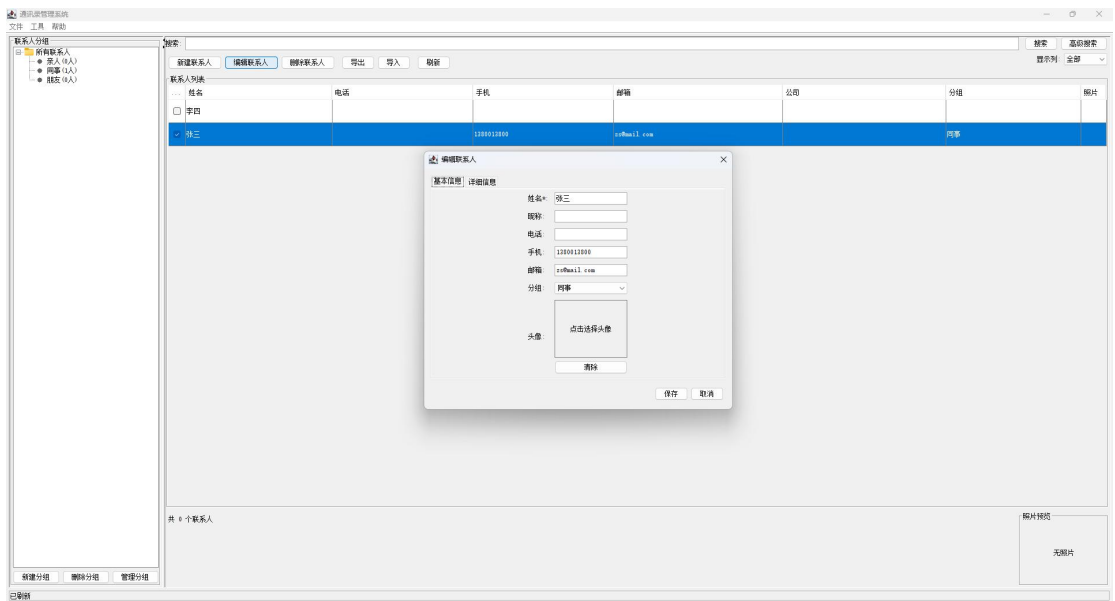


图 5-2

图 5-2 展示了联系人编辑对话框。对话框采用选项卡式布局，包含基本信息和详细信息两个标签页，支持照片选择和预览。

5.3 分组管理功能截图

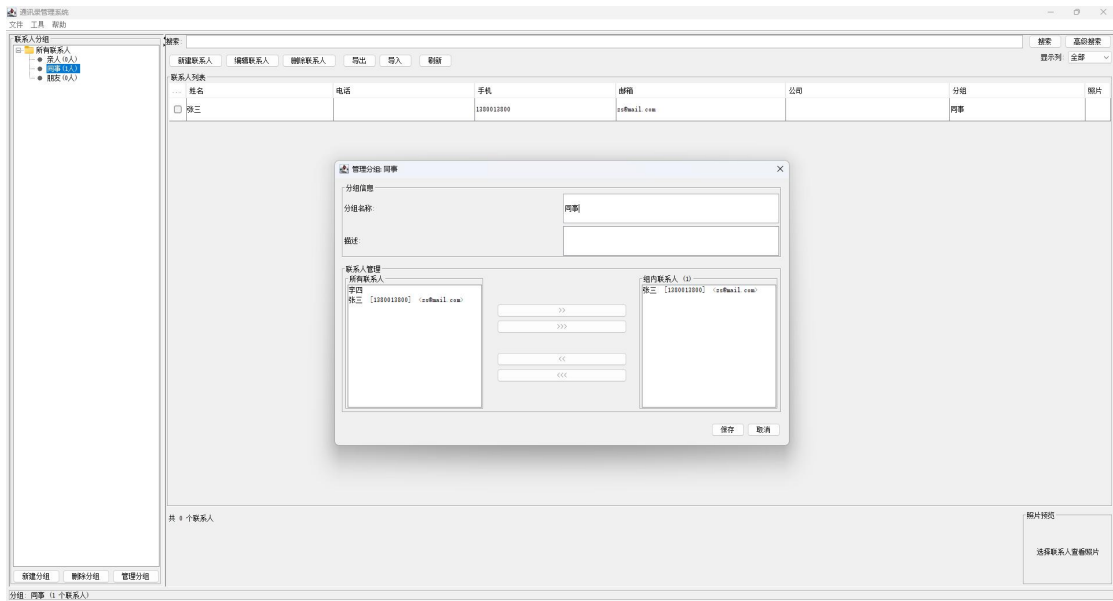


图 5-3

图 5-3 展示了分组管理对话框。对话框采用双列表设计，左侧为未分组联系人，右侧为分组内联系人，支持批量操作。

5.4 搜索功能截图

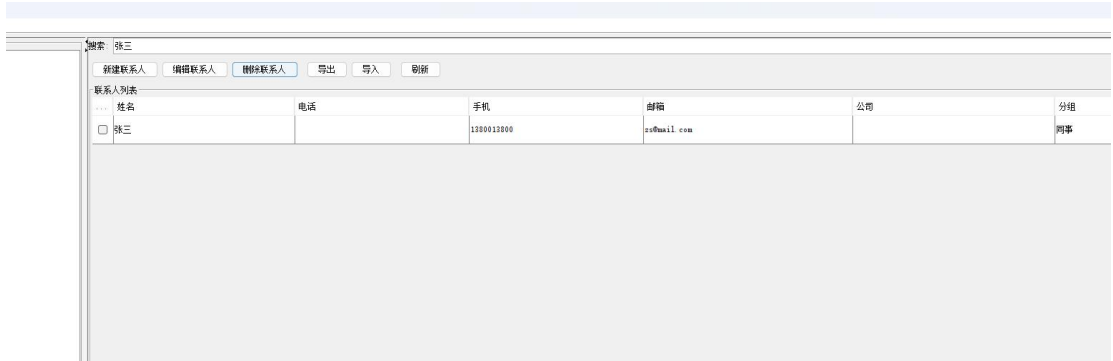


图 5-4

图 5-4 展示了搜索功能效果。在搜索框中输入关键词后，表格自动筛选显示匹配的联系。

5.5 导入导出功能截图

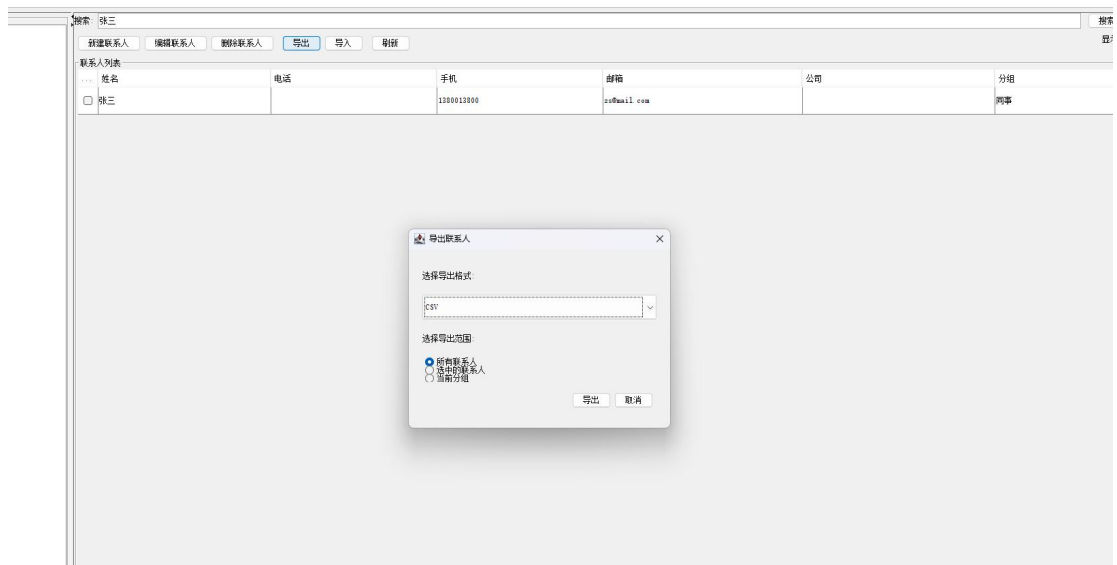


图 5-5

图 5-5 展示了导入导出对话框。支持 CSV 和 vCard 格式，可以选择导出全部、选中或当前分组的联系人。

5.6 数据备份恢复功能截图

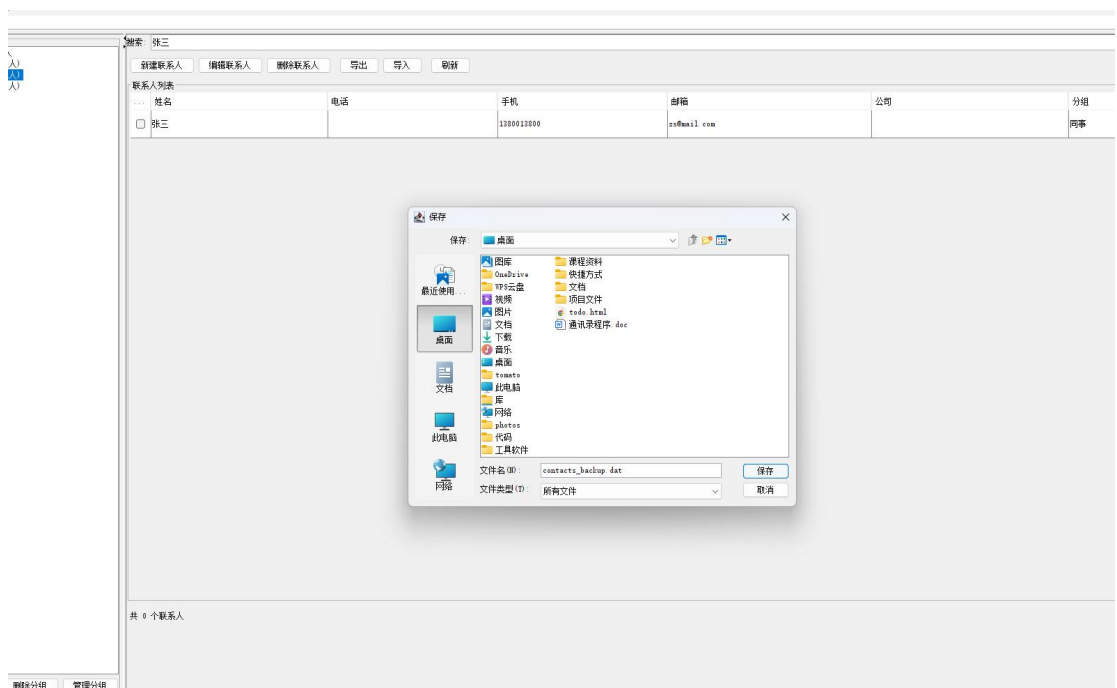


图 5-6

图 5-6 展示了备份恢复功能。支持将完整数据备份到指定位置，或从备份文件恢复数据。

六、 总结

陈凯文：

在本课程设计中，我主要负责导入导出模块、拼音搜索模块、照片管理模块以及核心模块的单元测试工作。具体包括：基于 pinyin4j 库实现汉字到拼音的转换，支持姓名全拼和拼音首字母的模糊搜索；实现 CSV 和 vCard 3.0 格式的导入导出功能，处理 UTF-8 编码、BOM 头、

特殊字符转义等问题，确保与主流手机通讯录的数据交换兼容；设计联系人照片的缩略图缓存机制和预览功能，优化界面响应速度；同时使用 JUnit 对文件持久化、拼音搜索、CSV 导入导出等核心逻辑编写单元测试，验证功能的正确性和数据一致性。

通过本次实践，我加深了对 Java 对象序列化、文件编码处理、正则表达式匹配以及单元测试框架的理解。在实现 vCard 解析时，遇到换行符和特殊字符转义的兼容性问题，通过查阅 RFC 2426 规范和多次测试得到了解决。照片缓存采用 LRU 策略，避免了内存溢出。团队协作中，我与刘坤龙同学保持了频繁的接口沟通，确保模型层与界面层的数据交互顺畅。这次课程设计不仅锻炼了我的编程能力，也让我体会到规范文档和充分测试对软件质量的重要性。

刘坤龙：

在本课程设计中，我主要负责界面交互模块、联系人管理模块、分组管理模块以及数据备份恢复模块的实现。具体包括：设计主窗口 MainFrame 的整体布局（左右分栏、工具栏、菜单栏、状态栏），实现联系人表格与分组树的联动交互；完成联系人的增删改查操作，支持单选及多选批量删除；实现分组的创建、删除以及成员调整（将联系人加入或移出分组）；开发数据的备份与恢复功能，通过文件流复制 contacts.dat 实现数据保护；实现表格显示列的动态切换（全部/基本信息/详细资料），方便查看不同维度的联系人信息；同时协调各模块的集成，尽量保证界面逻辑与业务逻辑的分离。

在开发过程中，我遇到的主要困难是 JTable 的实时刷新与表格模型同步问题，经过查阅资料和反复尝试，最终通过优化 DefaultTableModel 的数据刷新方式以及减少不必要的界面重绘，改善了界面响应问题。分组树中各分组人数的实时统计也是一个小难点，我通过监听联系人变更事件来触发树节点更新。备份恢复功能则让我注意到文件流的正确关闭和异常处理对数据安全性的重要性。通过本次课程设计，我对 Java Swing 桌面应用开发有了更系统的认识，也体会到了模块化设计和合理分工对团队协作的重要性。与陈凯文同学的合作过程中，我们定期交流进度、互相检查代码，最终顺利完成了功能比较完整的通讯录程序，是一次很有收获的实践经历。